

Simple Web Application Deployment on Render

Objective: To create a basic web application using Python (Flask) and deploy it to the cloud using Render.

Learning Outcomes: Upon completion of this exercise, you will be able to:

- Develop a simple web application using the Flask framework in Python.
- Understand the basic structure of a Flask application.
- Create necessary files for deploying a Python application on Render (e.g., `requirements.txt`).
- Deploy a Python web application to the Render cloud platform.
- Access your deployed web application via a public URL.

Requirements :

- A computer with Python 3 installed.
- A code editor (e.g., VS Code, Sublime Text, PyCharm).
- A free account on [Render](#).

Procedure :

Part 1: Creating the Flask Web Application

1. Create a Project Directory:

- o Open your terminal or command prompt.
- o Create a new directory for your project:

```
mkdir simple_web_app
cd simple_web_app
```

2. Create the Flask Application File (`app.py`):

- o Open your code editor and create a new file named `app.py` inside the `simple_web_app` directory.
- o Paste the following Python code into `app.py`:

Python code

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return "<h1>Hello from your Python Web App!</h1>"

if __name__ == '__main__':
    app.run(debug=True)
```

- o **Explanation:**
 - `from flask import Flask`: Imports the Flask class from the Flask library.
 - `app = Flask(__name__)`: Creates an instance of the Flask application. `__name__` helps Flask know where to look for static files and templates.
 - `@app.route('/')`: This decorator defines a route for the root URL (/). When a user accesses this URL, the `hello()` function will be executed.
 - `def hello()`: This function returns a simple HTML heading.
 - `if __name__ == '__main__':`: This ensures that the Flask development server runs only when the script is executed directly (not when imported as a module).
 - `app.run(debug=True)`: Starts the Flask development server. `debug=True` enables debugging mode, which is helpful during development but should be turned off in production.
- 3. **Create the `requirements.txt` File:**
 - o In the same `simple_web_app` directory, create a new file named `requirements.txt`.
 - o Add the following line to this file:
 - o `Flask`
 - o **Explanation:** This file lists the Python packages that need to be installed for your application to run. Render will use this file to install the necessary dependencies.

Part 2: Deploying to Render

1. **Sign Up for Render:**
 - o If you don't have one already, go to <https://render.com/> and sign up for a free account.
2. **Connect Your Repository (Optional but Recommended):**
 - o It's best practice to store your code in a Git repository (e.g., on GitHub, GitLab, or Bitbucket).
 - o On the Render dashboard, click on **New +** and select **Web Service**.
 - o Choose to connect your repository by selecting the provider (e.g., GitHub) and authorizing Render to access your repositories.
 - o Select the repository you created for this project (`simple_web_app`).
3. **Deploy Manually (If Not Using a Repository):**
 - o If you don't want to use a repository, you can still deploy by uploading your project files directly.
 - o On the Render dashboard, click on **New +** and select **Web Service**.
 - o Scroll down to the section "Deploy an existing repository" and look for an option like "Or deploy from a Git URL or deploy a static site." Choose the option that allows manual deployment (this might involve providing a public Git URL temporarily or other methods depending on Render's current interface).
4. **Configure Your Web Service:**
 - o **Name:** Give your web service a unique name (e.g., `simple-flask-app-yourusername`).
 - o **Region:** Choose a region that is geographically close to you.

- o **Branch (if using a repository):** Select the branch you want to deploy (usually `main` or `master`).
- o **Root Directory:** If your `app.py` and `requirements.txt` are in the root of your repository, leave this blank. If they are in a subdirectory, specify that path.
- o **Environment:** Select **Python 3**.
- o **Build Command:** Enter the command to install your dependencies:

```
pip install -r requirements.txt
```

- o **Start Command:** Enter the command to run your Flask application:

Bash

```
gunicorn app:app
```

- **Note:** We are using `gunicorn` here, which is a production-ready WSGI HTTP server. You'll need to add `gunicorn` to your `requirements.txt` file:
- `Flask`
- `gunicorn`

Make sure to update your `requirements.txt` and commit the change if you're using a repository.

- o **Instance Type:** For this simple application, the free tier should be sufficient.
 - o Click on **Create Web Service**.
5. **Wait for Deployment:**
- o Render will now build your application and deploy it. You can monitor the progress in the Render dashboard. This process might take a few minutes.
6. **Access Your Deployed Application:**
- o Once the deployment is successful, Render will provide you with a public URL (it will look something like `your-app-name.onrender.com`).
 - o Open this URL in your web browser. You should see the "Hello from your Python Web App!" message.

Part 3: Verification and Exploration

1. **Verify Deployment:** Ensure that your web application is accessible via the public URL provided by Render and that it displays the expected output.
2. **Explore Render Dashboard:** Familiarize yourself with the Render dashboard. You can see logs, manage deployments, configure settings, and more.

Further Exploration (Optional):

- Modify the `app.py` file to display more dynamic content or create multiple routes.
- Learn about serving static files (like CSS or images) with Flask and deploying those to Render.

- Explore environment variables in Render and how to use them in your Flask application.

Result:

This exercise provides a hands-on introduction to creating and deploying a simple Python web application to the cloud using Render.